

Application Security Verification Standard
(Mobile)



Table of Contents

1. MASVS-STORAGE: STORAGE	3
2. MASVS-CRYPTO: CRYPTOGRAPHY	3
3. MASVS-AUTH: AUTHENTICATION AND AUTHORIZATION	3
4. MASVS-NETWORK: NETWORK COMMUNICATION	3
5. MASVS-PLATFORM: PLATFORM INTERACTION	3
6. MASVS-CODE: CODE QUALITY	3
7. MASVS-RESILIENCE: RESILIENCE AGAINST REVERSE ENGINEERING AND TAMPERING	4
8. MASVS-PRIVACY: PRIVACY	4
MASVS-STORAGE	5
MASVS-CRYPTO	10
MASVS –AUTH AUTHENTICATION AND AUTHORIZATION	19
MASVS- NETWORK	28
MASVS-PLATFORM: PLATFORM INTERACTION	33
MASVS-CODE: CODE QUALITY	38
MASVS-RESILIENCE: RESILIENCE AGAINST REVERSE ENGINEERING AND TAMPERING	44
MASVS-PRIVACY	48

1. [MASVS-STORAGE: Storage](#)

- **MASVS-STORAGE-1:** The app securely stores sensitive data.
- **MASVS-STORAGE-2:** The app prevents leakage of sensitive data.

2. [MASVS-CRYPTO: Cryptography](#)

- **MASVS-CRYPTO-1:** The app employs current strong cryptography and uses it according to industry best practices.
- **MASVS-CRYPTO-2:** The app performs key management according to industry best practices.

3. [MASVS-AUTH: Authentication and Authorization](#)

- **MASVS-AUTH-1:** The app uses secure authentication, and authorization protocols and follows the relevant best practices.
- **MASVS-AUTH-2:** The app performs local authentication securely according to the platform best practices.
- **MASVS-AUTH-3:** The app secures sensitive operations with additional authentication.

4. [MASVS-NETWORK: Network Communication](#)

- **MASVS-NETWORK-1:** The app secures all network traffic according to the current best practices.
- **MASVS-NETWORK-2:** The app performs identity pinning for all remote endpoints under the developer's control.

5. [MASVS-PLATFORM: Platform Interaction](#)

- **MASVS-PLATFORM-1:** The app uses IPC mechanisms securely.
- **MASVS-PLATFORM-2:** The app uses WebViews securely.
- **MASVS-PLATFORM-3:** The app uses the user interface securely.

6. [MASVS-CODE: Code Quality](#)

- **MASVS-CODE-1:** The app requires an up-to-date platform version.
- **MASVS-CODE-2:** The app has a mechanism for enforcing app updates.
- **MASVS-CODE-3:** The app only uses software components without known vulnerabilities.
- **MASVS-CODE-4:** The app validates and sanitizes all untrusted inputs.

7. [MASVS-RESILIENCE: Resilience Against Reverse Engineering and Tampering](#)

- **MASVS-RESILIENCE-1:** The app validates the integrity of the platform.
- **MASVS-RESILIENCE-2:** The app implements anti-tampering mechanisms.
- **MASVS-RESILIENCE-3:** The app implements anti-static analysis mechanisms.
- **MASVS-RESILIENCE-4:** The app implements anti-dynamic analysis techniques.

8. [MASVS-PRIVACY: Privacy](#)

- **MASVS-PRIVACY-1:** The app minimizes access to sensitive data and resources.
- **MASVS-PRIVACY-2:** The app prevents identification of the user.
- **MASVS-PRIVACY-3:** The app is transparent about data collection and usage.
- **MASVS-PRIVACY-4:** The app offers user control over their data.

Note: Each section contains multiple checkpoints, and these checkpoints are taken from this [Link](#)

References

1. This standard is taken from the **OWASP** standard [link](#)
2. The Official Website [Link](#)
3. Master GitHub repository for Android: [Android](#)
4. Master GitHub repository for iOS: [IOS](#)
5. <https://mas.owasp.org/>

MASVS-STORAGE

- **MASVS-STORAGE-1:** The app securely stores sensitive data.
 1. Apps handle sensitive data coming from many sources such as the user, the backend, system services or other apps on the device and usually need to store it locally. The storage locations may be private to the app (e.g. its internal storage) or be public and therefore accessible by the user or other installed apps (e.g. public folders such as Downloads). This control ensures that any sensitive data that is intentionally stored by the app is properly protected independently of the target location.
- **MASVS-STORAGE-2:** The app prevents leakage of sensitive data.
 1. There are cases when sensitive data is unintentionally stored or exposed to publicly accessible locations; typically, as a side-effect of using certain APIs, system capabilities such as backups or logs. This control covers this kind of unintentional leaks where the developer has a way to prevent it.

MASVS-ID	Platform	Description
MASVS-STORAGE-1		The app securely stores sensitive data.
	android	Testing Local Storage for Sensitive Data
	android	Testing the Device-Access-Security Policy
	iOS	Testing Local Data Storage
MASVS-STORAGE-2		The app prevents leakage of sensitive data.
	android	Determining Whether Sensitive Data Is Shared with Third Parties via Embedded Services
	android	Determining Whether Sensitive Data Is Shared with Third Parties via Notifications
	android	Testing Backups for Sensitive Data
	android	Testing Memory for Sensitive Data
	android	Determining Whether the Keyboard Cache Is Disabled for Text Input Fields
	android	Testing Logs for Sensitive Data
	iOS	Checking Logs for Sensitive Data
	iOS	Testing Memory for Sensitive Data
	iOS	Testing Backups for Sensitive Data
	iOS	Finding Sensitive Data in the Keyboard Cache
	iOS	Determining Whether Sensitive Data Is Shared with Third Parties

Example Usage Guide

TESTING LOCAL STORAGE FOR SENSITIVE DATA

Standard Procedure: Android Local Data Storage Security Test

Objective: The purpose of this test is to identify sensitive information (such as passwords, keys, or personal data) that an Android application saves to the device and to verify that this data is stored securely. This procedure combines code review with hands-on testing of the live application.

Phase 1: Static Analysis (Code Review)

This initial phase involves analyzing the application's source code to identify potential data storage vulnerabilities before running the app.

- **Step 1: Review Application Permissions**

- **Action:** Analyze the AndroidManifest.xml file to identify any permissions related to external storage, such as android.permission.WRITE_EXTERNAL_STORAGE.
- **Bug Example:** The AndroidManifest.xml file requests the WRITE_EXTERNAL_STORAGE permission, but the application has no feature that requires it to save files to public storage. This is a bug because it creates an unnecessary risk that sensitive data could be written to a location accessible by any other app on the device.

- **Step 2: Identify Data Storage APIs**

- **Action:** Search the code for insecure file permissions like MODE_WORLD_READABLE and the usage of common storage APIs like SharedPreferences or getWritableDatabase.
- **Bug Example:** The code shows that the user's session token is being saved to SharedPreferences without any encryption. This is a bug because the token, which can be used to impersonate the user, is stored in a plain-text XML file on the device.

- **Step 3: Detect Insecure Encryption Practices**

- **Action:** Search the entire codebase for any hard-coded keys or passwords and verify that the secure Android KeyStore is being used for key management⁸.
- **Bug Example:** In a class file named DatabaseHelper.java, the encryption key for the entire user database is hard-coded as a string:

db.encrypt("MySecretPa\$\$w0rd"). This is a critical bug because anyone who can decompile the app can find this key and decrypt all user data.

- **Step 4: Check Common Secret Locations**

- **Action:** Inspect resource files like res/values/strings.xml and build configuration files like gradle.properties for sensitive data.
- **Bug Example:** The strings.xml file contains an entry: <string name="s3_api_key">AKI...789</string>. This is a bug because a secret API key is stored in plain text within the app's resources, making it easily extractable.

Phase 2: Dynamic Analysis (Live Application Testing)

This phase involves interacting with the running application to observe how it stores data in real-time.

- **Step 1: Prepare the Application Environment**

- **Action:** Install the application on a test device and use all of its features at least once.
- **Bug Example:** This step itself doesn't find a bug, but it creates the conditions for one to be found. For instance, a bug is found only *after* a user logs in: the app creates a file named session_details.txt in its data directory, which would not have existed otherwise.

- **Step 2: Inspect Application Storage Directories**

- **Action:** Check directories for SQLite databases, SharedPreferences XML files, and other files created by the app.
- **Bug Example:** After inspecting the /data/data/com.example.app/shared_prefs/ directory, a file is found containing <string name="user_password">password123</string>. This is a critical bug as it confirms the user's password is being stored in plain text on the device.

- **Step 3: Verify File Permissions**

- **Action:** Confirm that all files within the /data/data/<package-name> directory have secure permissions.
- **Bug Example:** A file containing user profile information is found to have its permissions set to be "world-readable". This is a bug because

it allows any other malicious application on the same phone to read the contents of this private file.

- **Step 4: Test for Insecure Cloud Databases**

- **Action:** If the app uses Firebase, check for misconfigurations by making a network call to `https://<firebaseProjectName>.firebaseio.com/.json`.
- **Bug Example:** Navigating to the app's Firebase URL in a web browser returns the entire user database in JSON format. This is a critical bug, indicating the database has been misconfigured with public read rules, exposing all user data to anyone on the internet.

References

- Testing Backups for Sensitive Data
- Testing the Device-Access-Security Policy
- Testing Backups for Sensitive Data
- [GitHub for Android](#)
- [GitHub for IOS](#)

MASVS-CRYPTO

MASVS-CRYPTO-1: Strong Cryptography Implementation

Checkpoint: The app employs current strong cryptography and uses it according to industry best practices.

Description: Cryptography is essential for protecting sensitive data on mobile devices, which are prone to theft or loss. The app must use modern, secure cryptographic algorithms and protocols, such as AES-256 for encryption and TLS 1.2 or higher for secure communications. These implementations should adhere to industry standards like NIST.SP.800-57 and NIST.SP.800-175B to ensure robustness against potential attacks.

MASVS-CRYPTO-2: Secure Key Management

Checkpoint: The app performs key management according to industry best practices.

Description: Even the strongest cryptographic algorithms can be compromised by poor key management. The app must securely generate, store, and manage cryptographic keys throughout their lifecycle. This includes using hardware-backed solutions like Android Keystore or iOS Secure Enclave, implementing key rotation policies, and ensuring keys are not hardcoded or exposed in the source code. Following guidelines from NIST.SP.800-57 Part 2 can help in establishing a secure key management framework

MASVS-ID	Platform	Description
<u>MASVS-CRYPTO-1</u>		The app employs current strong cryptography and uses it according to industry best practices.
	android	Testing Symmetric Cryptography
	android	Testing the Configuration of Cryptographic Standard Algorithms
	android	Testing Random Number Generation
	ios	Verifying the Configuration of Cryptographic Standard Algorithms
	ios	Testing Random Number Generation
<u>MASVS-CRYPTO-2</u>		The app performs key management according to industry best practices.
	android	Testing the Purposes of Keys
	ios	Testing Key Management

Example Usage Guide

This guide outlines static and dynamic analysis steps to verify compliance with MASVS-CRYPTO controls, focusing on strong cryptography implementation and secure key management.

Phase 1: Static Analysis (Code & Binary Review)

This phase involves examining the application's source code, compiled binaries, and configuration files without executing the application.

Targeting: MASVS-CRYPTO-1: Strong Cryptography Implementation

Objective: Ensure the app uses modern, secure cryptographic algorithms and protocols, adhering to industry best practices (e.g., AES-256, TLS 1.2/1.3, proper padding).

- **Step 1: Identify Cryptographic Primitives and Libraries**
 - **Action:**
 - Search the codebase for imports and instantiations of cryptographic classes (e.g., `javax.crypto.Cipher`, `java.security.MessageDigest`, `javax.net.ssl.SSLContext`, `android.security.keystore`).
 - Look for common cryptography-related keywords (AES, RSA, MD5, SHA, TLS, SSL, PKCS, GCM, CBC, ECB).
 - Analyze `build.gradle` (Android) or `Podfile`/`Gemfile`/`Package.swift` (iOS) for third-party cryptographic libraries.
 - **Bug Example:** The code instantiates `MessageDigest.getInstance("MD5")` for hashing sensitive data like passwords or `Cipher.getInstance("AES/ECB/PKCS5Padding")` for encrypting user data. This is a bug because MD5 is cryptographically broken for security purposes (collisions), and ECB mode is insecure for general-purpose encryption as it reveals patterns in the data.
- **Step 2: Verify TLS/SSL Configuration**
 - **Action:**

- Examine code related to network communication (e.g., `HttpsURLConnection`, `OkHttp` clients, `Alamofire`).
 - Check for `HostnameVerifier` or `SSLConnectionFactory` implementations, especially those that accept all certificates or disable hostname verification.
 - Look for `TrustManager` implementations that bypass certificate validation (`checkClientTrusted`, `checkServerTrusted`, `getAcceptedIssuers`).
 - Search for hardcoded trusted certificates or public keys (certificate pinning).
- **Bug Example:** The app's `TrustManager` implementation has an empty `checkServerTrusted` method or uses `HostnameVerifier.ALLOW_ALL_HOSTNAME_VERIFIER`. This is a critical bug as it makes the app vulnerable to Man-in-the-Middle (MitM) attacks, allowing an attacker to intercept and decrypt sensitive network traffic without being detected.
- **Step 3: Analyze Algorithm and Mode Usage**
 - **Action:**
 - For symmetric encryption, check `Cipher.getInstance()` arguments for algorithm, mode, and padding (e.g., ensure AES/GCM or AES/CBC with proper IVs, not ECB).
 - For hashing, verify that strong, collision-resistant algorithms (e.g., SHA-256, SHA-512) are used, especially for password storage (ideally with KDFs).
 - For asymmetric cryptography (RSA, ECC), check key lengths (e.g., RSA 2048-bit or higher) and appropriate padding schemes (e.g., OAEP for encryption, PSS for signatures).
 - **Bug Example:** The app uses `Cipher.getInstance("DES/CBC/PKCS5Padding")` for encrypting local user data. This is a bug because DES is an outdated and insecure block cipher with a small key size (56-bit), making it highly susceptible to brute-force attacks.

Targeting: MASVS-CRYPTO-2: Secure Key Management

Objective: Ensure cryptographic keys are securely generated, stored, and managed throughout their lifecycle, leveraging platform-provided secure storage.

- **Step 4: Detect Hardcoded Cryptographic Keys/Secrets**

- **Action:**

- Search the entire codebase (Java/Kotlin, Objective-C/Swift, C/C++ native code), resource files (res/values/strings.xml, Info.plist), and configuration files (gradle.properties, local.properties) for long strings that resemble cryptographic keys (hexadecimal, Base64), API keys, or passwords used as encryption keys.
 - Look for common variable names like `secretKey`, `encryptionKey`, `apiKey`, `password`.

- **Bug Example:** In a utility class, a line like `private static final String APP_ENCRYPTION_KEY = "mySuperSecretKey123ABC";` is found. This is a critical bug because the encryption key is directly embedded in the app's binary, making it trivial for an attacker to extract the key by decompiling the app and decrypt any data encrypted with it.

- **Step 5: Verify Key Storage Mechanisms**

- **Action:**

- Check how cryptographic keys (symmetric, asymmetric, password hashes) are stored.
 - Verify the use of platform-provided secure storage: Android Keystore (for Android) or Keychain Services / Secure Enclave (for iOS).
 - Look for instances where keys are stored insecurely (e.g., `SharedPreferences` without additional encryption, plain files in internal/external storage, `UserDefaults`, `NSUserDefaults`).

- **Bug Example:** The app saves an AES encryption key directly into `SharedPreferences` using `editor.putString("aes_key", base64EncodedKey)`. This is a bug because `SharedPreferences` data is stored in plain text XML files within the app's private directory, which can be accessed on a rooted device, exposing the encryption key.
- **Step 6: Analyze Key Generation and Derivation**
 - **Action:**
 - Examine how cryptographic keys are generated. Ensure cryptographically secure pseudo-random number generators (CSPRNGs) are used (e.g., `SecureRandom` in Java, `SecRandomCopyBytes` in iOS).
 - If keys are derived from user passwords, verify the use of strong Key Derivation Functions (KDFs) like PBKDF2, scrypt, or Argon2, with sufficient iterations/memory cost.
 - Check for the reuse of Initialization Vectors (IVs) or nonces with block ciphers like AES-CBC.
 - **Bug Example:** An encryption key is generated using `new Random().nextBytes(keyBytes)` or an IV is reused for every encryption operation. This is a bug because `Random` is not cryptographically secure, leading to predictable keys, and IV reuse in CBC mode makes the encryption vulnerable to chosen-plaintext attacks.

Phase 2: Dynamic Analysis (Live Application Testing & Runtime Inspection)

This phase involves interacting with the running application to observe its cryptographic behaviour and key management in real-time.

Targeting: MASVS-CRYPTO-1: Strong Cryptography Implementation

- **Step 1: Network Traffic Interception & TLS Analysis**
 - **Action:**

- Install the app on a test device/emulator and configure an intercepting proxy (e.g., Burp Suite, OWASP ZAP).
 - Attempt to proxy all app network traffic.
 - Inspect the TLS handshake details (protocol version, cipher suites, certificate chain, certificate validity).
 - Attempt to perform a Man-in-the-Middle (MitM) attack by using a self-signed root certificate.
- **Bug Example:** The app successfully communicates over HTTPS even when proxied with a self-signed certificate, without any warning or termination. The proxy logs show the TLS handshake using TLS 1.1 and weak cipher suites like TLS_RSA_WITH_RC4_128_SHA. This is a critical bug as it indicates a lack of proper certificate validation (or pinning) and the use of outdated/weak protocols/ciphers, making communications vulnerable to eavesdropping and manipulation.
- **Step 2: Runtime API Hooking for Crypto Functions**
 - **Action:**
 - Use dynamic instrumentation tools (e.g., Frida, Objection) to hook cryptographic API calls (Cipher.getInstance, SSLContext.init, KeyGenerator.getInstance, SecretKeySpec etc.).
 - Observe the parameters passed to these functions at runtime (e.g., algorithm names, key sizes, IVs, modes).
 - Intercept return values to see the generated keys or initialized ciphers.
 - **Bug Example:** After hooking Cipher.getInstance(), it's observed that the app is dynamically selecting AES/ECB/NoPadding based on some configuration. This confirms the static analysis finding of insecure mode usage, which might be dependent on runtime conditions or server-side configurations.

Targeting: MASVS-CRYPTO-2: Secure Key Management

- **Step 3: Inspect Application Storage Directories (Keys & Encrypted Data)**
 - **Action:**

- Root/jailbreak the test device.
 - Use adb shell (Android) or SSH (iOS) to navigate to the app's private data directories (/data/data/<package-name>, /var/mobile/Containers/Data/Application/<UUID>/Documents/ etc.).
 - List files and inspect their contents, looking for any files that might contain cryptographic keys in plain text or easily decryptable form.
 - Check shared_prefs XML files (Android) or UserDefaults plist files (iOS).
- **Bug Example:** A file named encryption_key.txt is found in the app's files directory, containing a plain text string that matches the encryption key used for local database encryption. This is a critical bug, confirming that a key is being stored in an easily accessible and unencrypted manner.
- **Step 4: Memory Analysis for Keys**
 - **Action:**
 - While the app is running and sensitive operations are performed (e.g., login, data encryption), perform a memory dump of the application's process.
 - Analyze the memory dump using forensic tools (e.g., strings command, specialized memory analysis tools) to search for cryptographic keys or unencrypted sensitive data that should be protected.
 - **Bug Example:** A memory dump of the app's process reveals the user's decrypted session token or the symmetric encryption key used for current communication, present in clear text in a non-volatile memory region (e.g., beyond the immediate scope of an operation). This is a bug because it shows improper handling of sensitive data or keys in memory, making them susceptible to cold boot attacks or other memory-scraping techniques.
- **Step 5: Test Key Protection in Secure Enclaves/Keystores**
 - **Action:**

- If the app claims to use hardware-backed keystores, attempt to extract keys directly from the Android Keystore or iOS Keychain/Secure Enclave via runtime tools or by forcing system-level backups (if applicable to the platform).
- Verify if user authentication (e.g., fingerprint, PIN) is enforced for key access where appropriate.
- **Bug Example:** A Frida script is able to successfully invoke the `getKey` method from the Android Keystore without requiring user authentication, even though the key's creation parameters specified `setUserAuthenticationRequired(true)`. This indicates a bypass or misconfiguration in how the key is accessed, compromising its hardware-backed protection.

References

- [Testing Symmetric Cryptography](#)
- [Testing Random Number Generation](#)
- [Testing Random Number Generation](#)
- [Testing the Purposes of Keys](#)
- [GitHub for Android](#)
- [GitHub for IOS](#)

MASVS –AUTH Authentication and Authorization

MASVS-AUTH-1: Secure Authentication and Authorization Protocols

Control: The app uses secure authentication, and authorization protocols and follows the relevant best practices.

Description: Mobile applications often require user authentication and enforce authorization to ensure that users can access only the resources they're permitted to. While the enforcement of these mechanisms should occur on the remote endpoint, the app must ensure it follows all relevant best practices to securely use the involved protocols. This includes implementing strong authentication methods, validating credentials properly, and managing user sessions securely.

MASVS-AUTH-2: Secure Local Authentication

Control: The app performs local authentication securely according to the platform best practices.

Description: Many mobile applications allow users to authenticate via biometrics or a local PIN code. These authentication mechanisms need to be correctly implemented to ensure security. Additionally, some apps might not have a remote endpoint and rely fully on local app authentication. In such cases, it's crucial to follow the platform's best practices for local authentication to safeguard user data and prevent unauthorized access.

MASVS-AUTH-3: Additional Authentication for Sensitive Operations

Control: The app secures sensitive operations with additional authentication.

Description: Certain sensitive actions within an app, such as changing account settings, making payments, or accessing confidential information, should require additional authentication beyond the initial login. This can be achieved through various methods, including biometric verification, PIN codes, multi-factor authentication (MFA), or email confirmations. Implementing such measures adds an extra layer of security, ensuring that even if an attacker gains access to the device, they cannot perform critical operations without proper authorization.

MASVS-ID	Platform	Description
<u>MASVS-AUTH-1</u>		The app uses secure authentication and authorization protocols and follows the relevant best practices.
<u>MASVS-AUTH-2</u>		The app performs local authentication securely according to the platform best practices.
	android	<u>TESTING BIOMETRIC AUTHENTICATION</u>
	android	<u>TESTING CONFIRM CREDENTIALS</u>
	ios	<u>TESTING LOCAL AUTHENTICATION</u>
<u>MASVS-AUTH-3</u>		The app secures sensitive operations with additional authentication.

Example Usage Guide

Developer Guidance: Always prioritize platform-provided secure APIs (Android BiometricPrompt with CryptoObject, iOS LocalAuthentication with Keychain Services). Avoid rolling your own biometric logic or blindly trusting third-party biometric SDKs without rigorous security vetting.

Phase 1: Static Analysis (Code & Configuration Review)

Review the app's source code, build configurations, and manifest files.

1. Check What Biometric Tools/APIs Are Used

- **iOS (Objective-C/Swift):**
 - **Action:** Analyze the app binary's shared dynamic libraries.
 - **Command:** `otool -L <AppName>.app/<AppName> | grep -E 'LocalAuthentication|Security'`
 - **Developer Action:** Confirm that `LocalAuthentication.framework` and `Security.framework` are linked. This indicates the app is using the system's biometric capabilities. If *only* `Security.framework` is present, it might indicate direct Keychain access, which is also relevant.
 - **What to Look For (Potential Bug):** If no biometric-related frameworks are linked but the app presents a biometric UI, it indicates a custom, likely insecure, or non-functional biometric implementation.
- **Android (Java/Kotlin):**
 - **Action:** Review `build.gradle` for `androidx.biometric:biometric` dependency. Search the codebase for `BiometricPrompt` class usage.
 - **Developer Action:** Ensure the app explicitly uses `androidx.biometric.BiometricPrompt` for biometric authentication, as `FingerprintManager` is deprecated and less secure.
 - **What to Look For (Potential Bug):** Usage of deprecated `FingerprintManager` or a custom biometric implementation not relying on `BiometricPrompt`.

2. Make Sure Biometric Is NOT The Only Protection (Cryptographic Binding)

- **Developer Principle:** Biometric authentication should primarily serve as a convenient way to **unlock access to a cryptographically protected secret (e.g., a session key, API token)**, rather than being the sole authentication mechanism for sensitive actions.
- **iOS Specifics:**
 - **Action:** Verify that sensitive information (e.g., session tokens, payment details, encryption keys) is stored securely in the **iOS Keychain**.
 - **Developer Action:** When adding items to the Keychain (SecItemAdd), ensure they are protected with appropriate Access Control Flags via SecAccessControlCreateWithFlags.
 - **Key Flags to Verify:**
 - kSecAccessControlBiometryCurrentSet (or kSecAccessControlTouchIDCurrentSet for older iOS versions): Requires biometrics and invalidates the entry if new fingerprints/faces are enrolled. **Highly Recommended for maximum security.**
 - kSecAccessControlBiometryAny (or kSecAccessControlTouchIDAny): Requires biometrics but allows new enrollments to access the item. Acceptable for convenience but less secure than CurrentSet.
 - kSecAccessControlUserPresence: Allows biometric *or* passcode fallback. **Considered weaker** due to potential for shoulder-surfing for passcodes. Use with caution for less sensitive data.
 - **Additional Protection:** Ensure kSecAttrAccessibleWhenPasscodeSetThisDeviceOnly or kSecAttrAccessibleWhenPasscodeSet is used to tie accessibility to the device passcode. The ThisDeviceOnly variant prevents iCloud synchronization.

- **What to Look For (Potential Bug):** Sensitive data stored in UserDefaults, NSUserDefaults, or unencrypted files. Keychain items stored with weak access control flags (e.g., kSecAttrAccessibleWhenUnlocked, kSecAttrAccessibleAlways), which don't require biometric authentication or allow access when the device is simply unlocked.
- **Android Specifics:**
 - **Action:** Verify that BiometricPrompt.authenticate() is always called with a BiometricPrompt.CryptoObject. This means the biometric scan is used to authenticate access to a hardware-backed key in the Android Keystore.
 - **Developer Action:** When generating keys in the Android Keystore for biometric use, ensure the KeyGenParameterSpec includes:
 - setUserAuthenticationRequired(true): Key requires user authentication to be used.
 - setUserAuthenticationValidityDurationSeconds(-1): Requires biometric authentication for *every* use of the key (no timeout grace period).
 - setInvalidatedByBiometricEnrollment(true): Invalidates the key if new biometrics are enrolled, providing stronger protection.
 - **What to Look For (Potential Bug):** Calling BiometricPrompt.authenticate() *without* a CryptoObject. Keys in Android Keystore not configured with setUserAuthenticationRequired(true) or with insecure validity durations.

3. Review How Biometric Failures/Exceptions Are Handled

- **Action:** Analyze the code within onAuthenticationError and onAuthenticationFailed callbacks (Android) or the error handling logic after evaluatePolicy (iOS).

- **Developer Action:** Ensure that in case of biometric authentication errors or failures, sensitive operations are *not* automatically allowed to proceed or fall back to an insecure method without proper re-authentication or a strong alternative.
- **What to Look For (Potential Bug):** Code that directly grants access or performs sensitive actions even after a biometric error, or that uses a weak fallback (e.g., a simple "OK" button) without re-validating the user.

Phase 2: Dynamic Analysis (Live Application Testing)

This phase involves interacting with the running application on a test device.

1. Prepare the Application Environment & Test Biometric Flow

- **Action:** Install the application on a rooted Android device/emulator or a jailbroken iOS device. Configure at least one biometric method (fingerprint, face). Exercise all app features that trigger biometric authentication.
- **Developer Action:** This step is preparation for testing. Ensure your test environment is properly set up for runtime analysis.

2. Test That Biometric Cannot Be Bypassed

- **Developer Principle:** A secure biometric implementation should resist runtime manipulation that attempts to bypass the biometric prompt without actual user authentication.

- **iOS Specifics:**

- **Action:** Use Objection's `ios ui biometrics_bypass` command.
- **Command:**

```
Bash
objection --gadget <AppBundleID> explore
ios ui biometrics_bypass
```

- **Expected Result (No Bug):** The Objection script *fails* to bypass the biometric authentication, or if it "succeeds" at the UI level, the app still

prevents access to the sensitive data or functionality because the underlying Keychain access failed.

- **What to Look For (Potential Bug):** The app's UI allows bypass, and sensitive data/functionality is accessible, indicating reliance solely on evaluatePolicy's boolean result without cryptographically binding to Keychain access.

- **Android Specifics:**

- **Action:** Use the provided Frida scripts (or similar from OWASP MASTG).
 - Fingerprint bypass: For apps *not* using CryptoObject.
 - Fingerprint bypass via exception handling: For apps *misusing* CryptoObject with insecure exception handling.

- **Commands:**

Bash

```
# For non-CryptoObject bypass:
frida -U -f <YOUR_APP_PACKAGE_NAME> --no-pause -l fingerprint-bypass.js

# For CryptoObject exception bypass:
frida -U -f <YOUR_APP_PACKAGE_NAME> --no-pause -l fingerprint-bypass-exception.js
```

- **Expected Result (No Bug):** The Frida scripts fail to bypass the biometric authentication, or even if they interfere, the app still *prevents access* to the sensitive data/functionality because the CryptoObject operation could not be completed securely.
- **What to Look For (Potential Bug):** The app grants access to sensitive functionality without a valid biometric scan when either script is active.

3. Check How Long Biometric Unlock Lasts (Android Specific)

- **Action:** After a successful biometric unlock, wait for the setUserAuthenticationValidityDurationSeconds period to expire (if

configured) or simply close and re-open the app. Try to access the biometric-protected feature again.

- **Developer Action:** Ensure that if `setUserAuthenticationValidityDurationSeconds` is set (e.g., to 30 seconds), the app requires re-authentication after that period. If set to -1, it should require re-authentication every time.
- **What to Look For (Potential Bug):** The app allows access to a biometric-protected feature without re-authentication, even after the validity duration should have expired, or the app was restarted. This indicates an insecure validity period or a failure to enforce re-authentication.

4. Sensitive Actions Should Always Require Re-authentication

- **Action:** For highly sensitive actions (e.g., confirming a payment, viewing critical PII, changing security settings), verify that the app always re-prompts for biometric authentication, even if a recent biometric unlock occurred for a less sensitive action.
- **Developer Action:** Implement "step-up authentication" where critical actions require a fresh biometric (or strong alternative) verification.
- **What to Look For (Potential Bug):** The app allows skipping biometric re-authentication for critical actions, relying on a previous, less sensitive, biometric unlock.

5. Review Any Third-Party Biometric SDKs

- **Action:** If third-party SDKs are used for biometric functionality, perform additional static and dynamic analysis on those SDKs where possible.
- **Developer Action:** Avoid relying on third-party SDKs that promise "biometric" functionality but don't explicitly document their use of platform-provided secure hardware (Keystore/Secure Enclave, Keychain Services with access controls). Always prefer native, platform-provided solutions.
- **What to Look For (Potential Bug):** Third-party SDKs that simply provide a UI prompt but don't involve cryptographic binding to secure keys, making

them vulnerable to bypasses. Known vulnerabilities in the specific SDK version.

References

- [TESTING CONFIRM CREDENTIALS](#)
- [TESTING BIOMETRIC AUTHENTICATION](#)
- [TESTING LOCAL AUTHENTICATION](#)
- [GITHUB FOR ANDROID](#)
- [GITHUB FOR IOS](#)

MASVS- NETWORK

MASVS-NETWORK-1: Secure Network Traffic

Control:

The app secures all network traffic according to the current best practices.

Description:

Apps must protect the **confidentiality and integrity** of data in transit. This is usually achieved using **TLS (Transport Layer Security)** to encrypt data and authenticate the server. Developers must ensure they do not disable secure defaults (e.g., using HTTP instead of HTTPS) or use insecure custom networking libraries. This control verifies that **all data transmitted over the network is properly encrypted and secured**, regardless of the API or library used.

MASVS-NETWORK-2: Identity Pinning

Control:

The app performs identity pinning for all remote endpoints under the developer's control.

Description:

Instead of trusting the entire list of root Certificate Authorities (CAs) bundled with the device or OS, **certificate pinning** ensures the app only trusts specific certificates or public keys. This mitigates **man-in-the-middle (MITM) attacks** and protects against compromised or rogue CAs. It's especially important for endpoints under the developer's control (e.g., app's backend servers).

MASVS-ID	Platform	Description
<u>MASVS-NETWORK-1</u>		The app secures all network traffic according to the current best practices.
	android	<u>TESTING ENDPOINT IDENTITY VERIFICATION</u>
	android	<u>TESTING THE SECURITY PROVIDER</u>
	android	<u>TESTING DATA ENCRYPTION ON THE NETWORK</u>
	android	<u>TESTING THE TLS SETTINGS</u>
	ios	<u>TESTING THE TLS SETTINGS</u>
	ios	<u>TESTING ENDPOINT IDENTITY VERIFICATION</u>
	ios	<u>TESTING DATA ENCRYPTION ON THE NETWORK</u>
<u>MASVS-NETWORK-2</u>		The app performs identity pinning for all remote endpoints under the developer's control.
	android	<u>TESTING CUSTOM CERTIFICATE STORES AND CERTIFICATE PINNING</u>
	ios	<u>TESTING CUSTOM CERTIFICATE STORES AND CERTIFICATE PINNING</u>

Example Usage Guide/Steps

1. Endpoint Identity Verification (MASTG-TEST-0021)

1.1. SDK Version & Network Security Configuration

- Ensure targetSdkVersion is ≥ 24 in build.gradle (or equivalent).
Reason: Android 7+ disables trust of user CAs by default.
- Verify network_security_config.xml does **NOT** include <certificates src="user" /> for production domains.
Reason: Prevents trusting user-installed CAs.
- Confirm android:usesCleartextTraffic="false" or omitted in <application> tag.
Reason: Blocks cleartext HTTP traffic by default on Android 9+.
- Confirm cleartextTrafficPermitted="false" for sensitive domains in <domain-config>.
Reason: Prevents cleartext overrides.
- **Tip:** Understand precedence rules for network security configs.

1.2. TLS Certificate Validation

- Review all custom TrustManager implementations: no trust-all shortcuts.
- Ensure cert validation covers chain, dates, CA trust, and no unintended self-signed certs.
- Confirm proper hostname verification, avoid HostnameVerifier that always returns true.
- Prefer using modern libraries with built-in TLS validation (OkHttp, Retrofit).

1.3. WebView TLS Handling

- WebViewClient.onReceivedSslError() should **NOT** call handler.proceed() silently in production.
- Confirm android:debuggable="false" in production manifest.

1.4. Dynamic Analysis - MITM Testing

- Test rejection of self-signed certs via proxy.

- Test rejection of untrusted CA certs (proxy-generated, no CA installed).
- Test rejection of hostname mismatches even with proxy CA installed.
- Check for certificate pinning; presence is a positive security sign.
- **Expected:** All invalid certs rejected with clear errors.

2. Security Provider Updates (MASTG-TEST-0023)

2.1. ProviderInstaller Usage

- Call `ProviderInstaller.installIfNeeded()` or `installIfNeededAsync()` early (e.g., in `Application.onCreate()`).
- Handle `GooglePlayServicesRepairableException` and `GooglePlayServicesNotAvailableException` properly.

2.2. Runtime Verification & Handling

- Detect if security provider update fails and alert user or restrict network-sensitive actions.
- Verify provider priority at runtime (`Security.getProviders()`).
- Keep Google Play Services updated on test devices.

3. TLS Version and Configuration (MASTG-TEST-0019)

3.1. Protocol Version Enforcement

- Ensure TLS 1.2 or higher is **enabled** and preferred for all network connections.
- Disable SSLv2, SSLv3, and TLS 1.0/1.1 unless explicitly required and justified.
- For custom SSL/TLS implementations (e.g., native libs or older HTTP clients), verify the enabled protocols list.

3.2. Cipher Suite Configuration

- Use strong cipher suites only (e.g., AES-GCM, ChaCha20-Poly1305).
- Avoid weak ciphers such as RC4, DES, 3DES, or NULL cipher suites.

3.3. TLS Session Configuration in Code

- When creating SSL sockets or using custom TLS configuration, explicitly specify TLS 1.2+ protocols and strong ciphers.
- Confirm no insecure fallback to SSL or older TLS versions.

3.4. Dynamic Testing and Verification

- Use tools (e.g., SSL Labs, custom interception proxies) to verify handshake protocols and cipher suites at runtime.
- Test network traffic to confirm no fallback to deprecated or vulnerable protocols/ciphers.

3.5. WebView TLS Configuration

- WebViews should use system default TLS versions or explicitly configured minimum TLS versions.

References

- [TESTING THE SECURITY PROVIDER](#)
- [TESTING DATA ENCRYPTION ON THE NETWORK](#)
- [TESTING ENDPOINT IDENTITY VERIFICATION](#)
- [TESTING CUSTOM CERTIFICATE STORES AND CERTIFICATE PINNING](#)
- [GITHUB FOR ANDROID](#)
- [GITHUB FOR IOS](#)

MASVS-PLATFORM: Platform Interaction

MASVS-PLATFORM-1: Secure IPC Mechanisms

Checkpoint:

The app uses Inter-Process Communication (IPC) mechanisms securely to control data and component exposure.

Description:

Mobile apps often rely on platform-provided IPC mechanisms such as Intents, Services, Broadcast Receivers (Android), or App Groups and URL Schemes (iOS) for communication. These should be implemented securely to prevent unauthorized access or misuse by other apps. Proper validation of incoming data, limiting component exposure using permissions or intent filters, and avoiding the use of implicit intents for sensitive actions are critical. Android components should explicitly declare their export settings and use permission enforcement. On iOS, custom URL schemes should validate and sanitize all incoming parameters to mitigate exploitation.

MASVS-PLATFORM-2: Secure Use of WebViews

Checkpoint:

The app uses WebViews in a secure manner to prevent unauthorized data access or code execution.

Description:

WebViews can introduce security risks when improperly configured, such as exposing JavaScript interfaces to untrusted content or allowing insecure content to load. The app must restrict WebView functionality to only what's necessary. JavaScript should be disabled unless absolutely required, and if used, interfaces like `addJavascriptInterface()` must be securely managed. Additionally, WebViews should be configured to load only content from trusted HTTPS sources, with file access and debugging features disabled in production. Input validation and URL filtering should be implemented to prevent injection attacks and malicious redirections.

MASVS-PLATFORM-3: Secure User Interface

Checkpoint:

The app implements user interface protections to prevent sensitive data exposure through screenshots, notifications, or shared views.

Description:

The mobile UI can be a source of unintended data leakage if not properly secured. Sensitive data like passwords, authentication tokens, or personal user details displayed in the UI should be protected against exposure. The app should utilize platform-specific flags (e.g., FLAG_SECURE on Android) to prevent screenshots or screen recording of sensitive screens. Additionally, apps must ensure that sensitive content is not shown in notifications unless the device is unlocked. Clipboard usage should be minimized for confidential data, and input fields should be properly masked. These measures help reduce the risk of data leaks through shoulder surfing, screen captures, or unintended sharing.

MASVS-ID	Platform	Description
MASVS-PLATFORM-1		The app uses IPC mechanisms securely.
	android	TESTING FOR APP PERMISSIONS
	android	TESTING FOR VULNERABLE IMPLEMENTATION OF PENDINGINTENT
	android	TESTING FOR SENSITIVE FUNCTIONALITY EXPOSURE THROUGH IPC
	android	DETERMINING WHETHER SENSITIVE STORED DATA HAS BEEN EXPOSED VIA IPC MECHANISMS
	android	TESTING DEEP LINKS
	ios	TESTING APP EXTENSIONS
	ios	TESTING CUSTOM URL SCHEMES
	ios	TESTING APP PERMISSIONS
	ios	DETERMINING WHETHER SENSITIVE DATA IS EXPOSED VIA IPC MECHANISMS
	ios	TESTING UIPASTEBOARD
	ios	TESTING FOR SENSITIVE FUNCTIONALITY EXPOSURE THROUGH IPC
	ios	TESTING UIACTIVITY SHARING

	ios	<u>TESTING UNIVERSAL LINKS</u>
<u>MASVS-PLATFORM-2</u>		The app uses WebViews securely.
	android	<u>TESTING WEBVIEW PROTOCOL HANDLERS</u>
	android	<u>TESTING WEBVIEWS CLEANUP</u>
	android	<u>TESTING JAVASCRIPT EXECUTION IN WEBVIEWS</u>
	android	<u>TESTING FOR JAVA OBJECTS EXPOSED THROUGH WEBVIEWS</u>
	ios	<u>TESTING WEBVIEW PROTOCOL HANDLERS</u>
	ios	<u>TESTING IOS WEBVIEWS</u>
	ios	<u>DETERMINING WHETHER NATIVE METHODS ARE EXPOSED THROUGH WEBVIEWS</u>
<u>MASVS-PLATFORM-3</u>		The app uses the user interface securely.
	android	<u>TESTING FOR OVERLAY ATTACKS</u>
	android	<u>CHECKING FOR SENSITIVE DATA DISCLOSURE THROUGH THE USER INTERFACE</u>
	android	<u>FINDING SENSITIVE INFORMATION IN AUTO-GENERATED SCREENSHOTS</u>
	ios	<u>CHECKING FOR SENSITIVE DATA DISCLOSED THROUGH THE USER INTERFACE</u>
	ios	<u>TESTING AUTO-GENERATED SCREENSHOTS FOR SENSITIVE INFORMATION</u>

Example Usage Guide/Steps

MASVS-PLATFORM-1: Secure IPC Mechanisms

- Ensure all components (Activities, Services, BroadcastReceivers) are marked `android:exported="false"` unless explicitly intended for use by other apps.
- Apply appropriate permissions (`android.permission`, `readPermission`, `writePermission`) to exported components and Content Providers.
- Use **explicit intents** for inter-component communication involving sensitive actions.
- Avoid using `PendingIntent` without proper flags (`FLAG_IMMUTABLE`, `FLAG_ONE_SHOT`).
- Validate all incoming data from Intents, Broadcasts, and Bound Services before processing.
- Avoid using implicit intents for actions that should only be triggered internally.
- For Content Providers, use parameterized queries and input validation to prevent SQL injection.
- Validate and sanitize data received from deep links and custom URL schemes.
- Do not expose sensitive functionality through unprotected IPC endpoints.

MASVS-PLATFORM-2: Secure WebView Usage

- Disable JavaScript in WebViews unless it is strictly required.
- If `addJavascriptInterface()` is used, ensure it is necessary and the interface is restricted using `@JavascriptInterface`.
- Do not load untrusted or user-supplied content into WebViews.
- Validate all URLs in `WebViewClient.shouldOverrideUrlLoading()` before loading.
- Disable file access in WebView settings:
 1. `setAllowFileAccess(false)`
 2. `setAllowFileAccessFromFileURLs(false)`
 3. `setAllowUniversalAccessFromFileURLs(false)`
- Ensure `WebView.setWebContentsDebuggingEnabled(false)` in production builds.

- Confirm android:debuggable="false" is set in the production AndroidManifest.xml.
- Clear WebView cache and history when sensitive content is no longer needed.
- Destroy WebView instance properly when not in use to release resources.

MASVS-PLATFORM-3: Secure User Interface

- Use FLAG_SECURE to prevent screenshots and screen recording on screens displaying sensitive data.
- Mask sensitive fields in the UI (e.g., passwords, OTPs, credit card numbers).
- Disable clipboard copy for sensitive data where possible.
- If clipboard is used for temporary data (e.g., OTP), clear it promptly after use.
- Avoid placing sensitive information in push notifications or use private visibility settings.
- Use Notification.VISIBILITY_PRIVATE to control notification content display on locked screens.
- Detect and handle potential overlay attacks (e.g., during sensitive UI interactions).
- Prevent auto-fill/autocomplete for sensitive input fields.
- Validate that sensitive UI data is not present in auto-generated screenshots or logs.

References

- [**DETERMINING WHETHER SENSITIVE STORED DATA HAS BEEN EXPOSED VIA IPC MECHANISMS**](#)
- [**TESTING APP EXTENSIONS**](#)
- [**TESTING CUSTOM URL SCHEMES**](#)
- [**TESTING AUTO-GENERATED SCREENSHOTS FOR SENSITIVE INFORMATION**](#)
- [**GITHUB FOR ANDROID**](#)
- [**GITHUB FOR IOS**](#)

MASVS-CODE: Code Quality

MASVS-CODE-1: The app requires an up-to-date platform version

Checkpoint: Ensure the app targets the latest stable platform version and does not support outdated OS versions.

Description:

Every release of the mobile OS includes critical security patches and new features aimed at improving security. By supporting older versions of the platform, an app may remain vulnerable to known security threats. This control ensures that the app is running on an up-to-date platform version, minimizing the risk of exploitation through known vulnerabilities.

MASVS-CODE-2: The app has a mechanism for enforcing app updates

Checkpoint: Ensure that the app checks for and enforces updates automatically or prompts the user to update if a new version is available.

Description:

Critical vulnerabilities in apps can be discovered after they are released. If not addressed in a timely manner, these vulnerabilities can be exploited. This control ensures that there is a mechanism in place that forces users to update the app before they can continue using it, minimizing the window of exposure to known threats. This can be achieved by incorporating forced update features or checking for newer versions during runtime.

MASVS-CODE-3: The app only uses software components without known vulnerabilities

Checkpoint: Use tools or services (e.g., OWASP Dependency-Check, Snyk, or other vulnerability scanners) to verify third-party libraries and dependencies are free of known vulnerabilities.

Description:

Using third-party software components is common in app development, but it's essential to ensure that none of these components have known vulnerabilities. This control mandates that the app only uses software components that do not have known

security flaws. Vulnerability scanning tools can help identify outdated libraries or components that may contain exploitable vulnerabilities. Ideally, a full white-box assessment should be performed, but for most practical purposes, scanning for known issues is a baseline requirement.

MASVS-CODE-4: The app validates and sanitizes all untrusted inputs

Checkpoint: Ensure that any data coming from untrusted sources (e.g., user input, network data, IPC, etc.) is validated and sanitized before being processed by the app.

Description:

Untrusted input is a common vector for attacks such as SQL injection, Cross-Site Scripting (XSS), and insecure deserialization. Apps must validate all incoming data—whether from users, external devices, or network connections—to ensure it adheres to expected formats, ranges, and types. Sanitizing inputs ensures that any harmful data, such as script code or malicious payloads, is removed or neutralized. This control helps protect against many common attack vectors that rely on manipulating untrusted input data.

MASVS-ID	Platform	Description
<u>MASVS-CODE-1</u>		The app requires an up-to-date platform version.
<u>MASVS-CODE-2</u>		The app has a mechanism for enforcing app updates.
	android	<u>Testing Enforced Updating</u>
	ios	<u>Testing Enforced Updating</u>
<u>MASVS-CODE-3</u>		The app only uses software components without known vulnerabilities.
	android	<u>Checking for Weaknesses in Third Party Libraries</u>
	ios	<u>Checking for Weaknesses in Third Party Libraries</u>
<u>MASVS-CODE-4</u>		The app validates and sanitizes all untrusted inputs.
	android	<u>Make Sure That Free Security Features Are Activated</u>
	android	<u>Testing for Injection Flaws</u>
	android	<u>Testing Local Storage for Input Validation</u>
	android	<u>Memory Corruption Bugs</u>
	android	<u>Testing Object Persistence</u>
	android	<u>Testing Implicit Intents</u>
	android	<u>Testing for URL Loading in WebViews</u>
	ios	<u>Testing Object Persistence</u>
	ios	<u>Memory Corruption Bugs</u>
	ios	<u>Make Sure That Free Security Features Are Activated</u>

Example Usage Guide/Steps

MASTG-TEST-0036: Testing Enforced Updating

Static Analysis Checkpoints:

- **Add support for in-app updates:** Use `AppUpdateManager` to check for updates when the app starts. Make sure it calls `getAppUpdateInfo()` to check if an update is available.
- **Force immediate update:** If an update is available, use `AppUpdateType.IMMEDIATE` to make sure users cannot skip the update.
- **Start the update flow:** Once an update is available, call `startUpdateFlowForResult()` to show the update dialog.
- **Handle update result:** Implement `onActivityResult()` to handle success or failure of the update (e.g., retry if update fails).
- **Check for updates during app resume:** In `onResume()`, make sure to check if the update is already in progress and continue it if needed.

Dynamic Analysis Checkpoints:

- **Test with an outdated app version:** Install an older version of the app and confirm that it forces an update before continuing.
- **Try bypassing update prompt:** Cancel the update prompt to see if the app blocks usage until the update is done.
- **Modify app version:** Use a MITM proxy to modify the version number and see if the backend blocks outdated versions.

MASTG-TEST-0042: Checking for Weaknesses in Third Party Libraries (Deprecated)

Static Analysis Checkpoints:

- **Use Dependency Checker:** Integrate the **OWASP Dependency-Check Gradle plugin** to check for vulnerabilities in third-party libraries.
- **Review dependency report:** After running the tool, review the report for any vulnerable libraries and check their versions.

- **Update or replace vulnerable libraries:** If vulnerabilities are found, update the library to a patched version or find a secure alternative.

Dynamic Analysis Checkpoints:

- **Verify third-party library licenses:** Ensure that you include copyright and license information in the app (usually in an "About" section or EULA).

MASTG-TEST-0025: Testing for Injection Flaws

Static Analysis Checkpoints:

- **Check for exposed IPC mechanisms:** Review Android's Manifest.xml for components like content providers, services, or deep links that could be used by other apps (or attackers).
- **Sanitize all input:** Ensure that input, especially from untrusted sources (e.g., IPC, URL schemes), is validated. For example, input should only be numeric or alphanumeric when expected.
- **Check for SQL injection risk:** Make sure that SQL queries (especially in content providers) use prepared statements, not raw concatenation, to prevent SQL injection.

Dynamic Analysis Checkpoints:

- **Test for SQL injection:** Try entering dangerous strings like OR 1=1-- in input fields to check if SQL injection vulnerabilities exist.
- **Test IPC interactions:** If the app uses content providers, services, or deep links to exchange data, check that it sanitizes inputs properly to prevent injection attacks.

MASTG-TEST-0043: Memory Corruption Bugs

Static Analysis Checkpoints:

- **Look for unsafe memory handling in native code:** If your app uses native code (C++, NDK), check for memory issues like buffer overflows or improper memory access.
- **Check for memory leaks in Java/Kotlin code:** Ensure that objects, especially Activity and Context, are not held by static references. Unregister BroadcastReceivers when they're no longer needed.
- **Verify proper deserialization:** When the app receives data that needs deserialization (e.g., JSON), make sure it is done safely without exposing the app to vulnerabilities.

Dynamic Analysis Checkpoints:

- **Check for memory leaks:** Use tools like **LeakCanary** or **Android Studio's Memory Profiler** to catch memory leaks during runtime.
- **Analyze native code memory usage:** If using native code, check memory usage with **Valgrind** or similar tools to catch any issues.
- **Test for unsafe deserialization:** Use tools like **Android Java Deserialization Vulnerability Tester** to check for insecure deserialization.

References

- [Testing Enforced Updating](#)
- [Checking for Weaknesses in Third Party Libraries](#)
- [Memory Corruption Bugs](#)
- [Testing Object Persistence](#)
- [Checking for Weaknesses in Third Party Libraries](#)
- [GitHub for Android](#)
- [GitHub for IOS](#)

MASVS-RESILIENCE: Resilience Against Reverse Engineering and Tampering

1. MASVS-RESILIENCE-1: Platform Integrity Validation

- a. **Description:** The app should verify that it is running on a trusted, uncompromised platform (i.e., not rooted or using a custom ROM). This is critical to ensure the reliability of core security features like secure storage, biometrics, and sandboxing.

2. MASVS-RESILIENCE-2: Anti-Tampering Mechanisms

- a. **Description:** The app should detect and prevent modifications to its code or resources. This helps block attackers from altering app logic, enabling cheats, removing security checks, or repackaging the app with malicious functionality.

3. MASVS-RESILIENCE-3: Anti-Static Analysis Protections

- a. **Description:** The app should include measures (like obfuscation) to make it harder for attackers to understand the code via static analysis tools. This reduces the risk of reverse engineering and discovering sensitive logic.

4. MASVS-RESILIENCE-4: Anti-Dynamic Analysis Techniques

- a. **Description:** The app should resist runtime inspection and manipulation (e.g., via Frida or hooking). Techniques include detecting debugging, emulator use, or dynamic instrumentation to prevent tampering or sensitive data extraction.

MASVS-ID	Platform	Description
<u>MASVS-RESILIENCE-1</u>		The app validates the integrity of the platform.
	android	Testing Root Detection
	android	Testing Emulator Detection
	ios	Testing Jailbreak Detection
	ios	Testing Emulator Detection
<u>MASVS-RESILIENCE-2</u>		The app implements anti-tampering mechanisms.
	android	Testing File Integrity Checks
	android	Testing Runtime Integrity Checks
	android	Making Sure that the App is Properly Signed
	ios	Testing File Integrity Checks
	ios	Making Sure that the App Is Properly Signed
<u>MASVS-RESILIENCE-3</u>		The app implements anti-static analysis mechanisms.
	android	Testing for Debugging Symbols
	android	Testing for Debugging Code and Verbose Error Logging
	android	Testing Obfuscation
	ios	Testing for Debugging Code and Verbose Error Logging
	ios	Testing Obfuscation
	ios	Testing for Debugging Symbols
<u>MASVS-RESILIENCE-4</u>		The app implements anti-dynamic analysis techniques.
	android	Testing whether the App is Debuggable
	android	Testing Reverse Engineering Tools Detection
	android	Testing Anti-Debugging Detection
	ios	Testing Anti-Debugging Detection
	ios	Testing whether the App is Debuggable
	ios	Testing Reverse Engineering Tools Detection

Example Usage Guide/Steps

1. Root Detection (Android)

Objective: Ensure the app detects and refuses to run on rooted devices.

- Implement multiple root detection techniques (e.g., checking for su, root-only apps, test-keys).
- Perform checks at both Java and native layers.
- Avoid using publicly known or copy-pasted detection methods.
- Test resistance by bypassing with tools like RootCloak, Frida, or MagiskHide.
- Assess if root detection can be trivially bypassed or if it requires advanced techniques.

2. Jailbreak Detection (iOS)

Objective: Prevent execution on jailbroken iOS devices.

- Implement detection for common jailbreak indicators (e.g., presence of Cydia, suspicious file paths).
- Validate behavior on jailbroken vs. non-jailbroken devices.
- Attempt to bypass detection with jailbreak hiding tools.
- Reverse engineers your own implementation to assess ease of detection bypass.

3. File Integrity Checks (Android and iOS)

Objective: Detect tampering with application files and sensitive data.

- Validate the integrity of code files (e.g., classes.dex, .so, or IPA executables).
- Protect data files using HMACs or cryptographic signatures.
- Test detection by modifying files and repackaging the app.
- Simulate storage tampering (e.g., edited preferences or DB) and confirm detection is triggered.
- Evaluate whether the mechanisms can be bypassed using hooking or simple patching.

4. Code Obfuscation (Android)

Objective: Make reverse engineering difficult by obfuscating code.

- Use obfuscation tools (e.g., ProGuard, R8) to rename classes, methods, and fields.
- Encrypt sensitive strings and API endpoints.
- Apply native-level obfuscation for native code (e.g., Obfuscator-LLVM).
- Analyze the APK using tools like JADX or APKID to confirm effectiveness of obfuscation.

5. Anti-Debugging (Android)

Objective: Prevent app debugging through both static and dynamic means.

- Implement detection for debugging (e.g., `isDebuggerConnected()`, `ptrace`).
- Integrate checks across both Java and native layers.
- Avoid centralized logic—spread detection throughout the app.
- Attempt to attach debuggers and observe app behavior (e.g., app exits or malfunctions).
- Measure difficulty of bypassing mechanisms using Frida or patching techniques.

References

- <https://mas.owasp.org/MASTG/tests/android/MASVS-RESILIENCE/MASTG-TEST-0046/>
- <https://mas.owasp.org/MASTG/tests/android/MASVS-RESILIENCE/MASTG-TEST-0050/>
- <https://mas.owasp.org/MASTG/tests/ios/MASVS-RESILIENCE/MASTG-TEST-0093/>
- <https://mas.owasp.org/MASTG/tests/ios/MASVS-RESILIENCE/MASTG-TEST-0091/>
- [GitHub for Android](#)
- [GitHub for IOS](#)

MASVS-PRIVACY

MASVS-PRIVACY-1

Control: The app minimizes access to sensitive data and resources.

Checklist:

- App only requests permissions essential for core features (data minimization).
- Access to sensitive data (e.g., location, camera, contacts) is preceded by user consent.
- User is informed about the purpose of each permission request.
- Third-party SDKs are not allowed to access data before consent is explicitly granted.
- SDKs are configured to comply with user consent and do not collect data unnecessarily.
- Data is not shared with SDKs in the supply chain unless required and disclosed.
- SBOM (Software Bill of Materials) is maintained to identify third-party data access dependencies.

MASVS-PRIVACY-2

Control: The app prevents identification of the user.

Checklist:

- App uses pseudonymization or anonymization techniques where possible.
- Use of persistent identifiers (e.g., device ID, IMEI) is minimized or avoided entirely.
- Technical controls ensure separation of fingerprinting data for distinct purposes (e.g., fraud vs analytics).
- Fingerprinting techniques (e.g., behavior tracking, IP logging) are documented, scoped, and not reused across domains.
- No cross-context linkage of user identifiers unless clearly justified and consented.

MASVS-PRIVACY-3

Control: The app is transparent about data collection and usage.

Checklist:

- The app provides a clear, concise privacy notice aligned with its actual data practices.

- Privacy policy includes: data types collected, purpose, storage, sharing, and retention.
- Unexpected behaviors (e.g., background data access) are explicitly disclosed.
- Any changes in data collection after updates are reflected in updated disclosures.
- Platform-specific declarations (e.g., Google Play Data Safety Section) are accurate and kept current.

MASVS-PRIVACY-4

Control: The app offers user control over their data.

Checklist:

- Users can view, export, delete, or modify their data within the app or through account settings.
- App provides privacy settings for users to enable/disable data collection and revoke consent.
- Re-consent is requested whenever the scope of data collection changes.
- Requests for account or data deletion are honored within a defined and documented timeframe.
- Consent records are securely logged and managed to maintain auditability.

MASVS-ID	Platform	Description / MASTG Test
MASVS-PRIVACY-1		The app minimizes access to sensitive data and resources.
	Android	Testing Dangerous App Permissions
	Android	Testing Permission Requests Not Minimized
	Android	Testing Missing Permission Rationale
	Android (11+)	Testing Not Resetting Unused Permissions
MASVS-PRIVACY-2		The app prevents identification of the user.
	Android / iOS	Testing Undeclared Known Tracking Domains
MASVS-PRIVACY-3		The app is transparent about data collection and usage.
MASVS-PRIVACY-4		The app offers user control over their data.

References

1. <https://owasp.org/www-project-mobile-app-security/>
2. <https://mas.owasp.org/MASVS/12-MASVS-PRIVACY/#controls>
3. <https://mas.owasp.org/checklists/MASVS-PRIVACY/>